

Compile-Evaluate-Prune (CEP)

The Compiler as Zero-Cost Oracle: Unified Pruning and Architecture Search Through Compilation Analysis in NeuralScript

Authors: Brandon Wiemer

Date: April 2026

Status: Research Proposal

Abstract

We propose **Compile-Evaluate-Prune (CEP)**, a unified framework for structured pruning and hardware-aware architecture search where the NSL compiler itself serves as the evaluation oracle. In existing frameworks, pruning requires runtime importance estimation (gradient-based, activation-based, or Hessian-based), and NAS requires either training thousands of candidate architectures or building surrogate models to approximate performance. CEP replaces both with **compilation analysis**: the compiler loads pre-trained weights, compiles a candidate architecture, and produces exact hardware performance metrics (FLOPs, HBM traffic, latency, peak memory, roofline utilization) in milliseconds — without training a single step. Critically, because NSL compiles to native code and generates its own PTX, a structurally pruned model doesn't just have zeroed weights — the pruned computation is *physically absent* from the binary. The compiled program is smaller, uses fewer registers, launches smaller kernels, and consumes less memory than the unpruned version. This is the "true pruning" that PyTorch's zeroed-weight approach only approximates.

1. The Problem: Pruning and NAS are Evaluation-Bottlenecked

1.1 Pruning's Evaluation Problem

Structured pruning removes entire components (heads, neurons, layers) from a model. The core question is: *which components should be removed?* Every existing method answers this at runtime:

- **Magnitude pruning:** Compute weight norms → remove smallest. Requires loading all weights and computing norms at runtime.
- **Gradient-based (LLM-Pruner, CFSP):** Run calibration data forward+backward → measure gradient importance per structure. Requires actual GPU computation.
- **Activation-based (Wanda):** Run calibration forward passes → measure activation magnitudes. Requires forward passes.
- **Hessian-based (OBS, GPTQ-style):** Compute or approximate the Hessian → measure sensitivity. Extremely expensive.

After pruning, the model must be **restructured** — weight matrices reshaped, layer connections updated, attention head counts adjusted. In PyTorch, this means rewriting `nn.Module` code or using complex hooks. The restructured model then needs separate profiling to verify it meets hardware targets.

1.2 NAS's Evaluation Problem

Hardware-aware NAS evaluates candidate architectures on two criteria: task performance (accuracy/loss) and hardware efficiency (latency/memory). Current approaches:

- **Supernet training:** Train a weight-sharing supernet, then extract sub-networks. Cost: GPU-days.
- **Zero-cost proxies:** Use cheap metrics (SynFlow, SNIP, Jacobian covariance) to predict task performance without training. These are approximate and often poorly correlated with actual performance for LLM architectures.
- **Surrogate models:** Train a small regression model (MLP, XGBoost) to predict latency from architecture parameters. Requires hardware profiling data for each target device.

The fundamental bottleneck: No existing framework can cheaply and exactly evaluate both task-performance impact AND hardware efficiency for a given architectural change. Zero-cost proxies approximate the former; latency lookup tables approximate the latter. Neither is exact.

1.3 NSL's Unique Position

NSL can do something no other framework can: **compile a candidate architecture and get exact hardware metrics in milliseconds**. The compiler's cost model (M37) provides per-op FLOP counts, HBM traffic, arithmetic intensity, and roofline-bounded latency estimates. The memory planner (M36) provides exact peak memory. The WCET analyzer (M53) provides worst-case latency bounds. All of these run on the AST — no GPU required.

Furthermore, when weights are available (`ns1 build --weights`), the compiler can perform weight-aware analysis (M52): spectral decomposition, sparsity detection, effective rank estimation, and constant folding through pruned paths.

CEP unifies pruning and NAS by treating both as **architecture search problems evaluated through compilation**.

2. CEP Architecture

2.1 The Compilation Oracle

The CEP oracle takes a model definition + optional pre-trained weights and returns a multi-objective evaluation vector without any training:

Input:

- Model AST (from .nsl source)
- Target hardware spec (from GPU database)
- Pre-trained weights (optional, from .safetensors)
- Calibration data (optional, small sample)

Compilation Analysis Pipeline:

1. Shape propagation → exact tensor shapes at every layer
2. FLOP counting → per-op and total forward FLOPs
3. Roofline analysis → per-op compute/memory bound classification
4. HBM traffic model → bytes read/written per op
5. Memory planning → peak activation memory, parameter memory
6. Latency estimation → roofline-bounded time per op
7. Fusion analysis → which ops will fuse, adjusted metrics
8. [If weights] Weight analysis → per-head/neuron importance scores

Output: CompilationProfile

- total_flops: u64
- total_hbm_bytes: u64
- peak_memory_bytes: u64
- estimated_latency_us: f64
- per_layer_profiles: Vec<LayerProfile>
- per_head_importance: Vec<f64> (if weights provided)
- roofline_utilization: f64
- fusion_opportunities: Vec<FusionEvent>

Cost of one oracle evaluation: ~50ms for a 50M parameter model (AST traversal + arithmetic).

Compare to:

- Zero-cost proxy (SynFlow): ~1 second (requires forward pass)
- One training step: ~100ms (GPU compute)
- Full NAS evaluation: hours to days

2.2 Three Operating Modes

Mode 1: Compile-Time Pruning (weights available)

Given a pre-trained model and target hardware, CEP determines the optimal pruning configuration:

```
Input: model.nsl + model.safetensors + target=rtx5070ti + sparsity=50%

CEP Pruning Pipeline:
1. Compile original model → baseline CompilationProfile
2. Weight analysis:
  a. Per-head importance:  $\|W_{q,h}\|_F \times \|W_{k,h}\|_F$  (attention contribution)
  b. Per-neuron importance:  $\|w_{in}\|_1 \times \|w_{out}\|_1$  (FFN contribution)
  c. Per-layer importance: gradient-free sensitivity via weight magnitude
3. Generate candidate pruned architectures (combinatorial):
  - Remove head  $h$  from layer  $l \rightarrow$  recompile  $\rightarrow$  evaluate
  - Remove neuron  $n$  from FFN  $l \rightarrow$  recompile  $\rightarrow$  evaluate
  - Remove entire layer  $l \rightarrow$  recompile  $\rightarrow$  evaluate
4. Multi-objective optimization:
  - Maximize: retained capacity (sum of importance scores)
  - Minimize: latency (from compilation oracle)
  - Constraint: peak memory  $\leq$  budget, sparsity  $\geq$  target
5. Output: Pareto-optimal pruning configurations
```

Why compilation matters for pruning evaluation: When you remove 2 of 8 attention heads, the Q/K/V projection weight matrices shrink from $[D, D]$ to $[D, D \times 6/8]$. This changes:

- The matmul tile sizes (cost model recalculation)
- The FlashAttention kernel configuration (6 heads vs 8 — different grid dimensions)
- The shared memory layout (smaller Q/K/V tiles)
- The fusion opportunities (smaller matmuls may now be fusible with adjacent ops)
- The memory plan (different liveness patterns)

PyTorch pruning tools compute none of this — they measure importance and zero out weights, but the model still runs the same-sized kernels. NSL recompiles with the new dimensions, producing kernels exactly sized for the pruned architecture.

Mode 2: Compile-Time Architecture Search (no weights)

Given a model template with configurable parameters, CEP searches for the hardware-optimal configuration:

ns!

```
# Model template with searchable parameters
@cep_search(
    d_model = [256, 384, 512, 768],
    n_layers = [4, 6, 8, 12],
    n_heads = [4, 6, 8],
    n_kv_heads = [2, 3, 4],
    d_ff_ratio = [2.0, 2.67, 4.0],
    target = rtx5070ti,
    constraints = { peak_memory < 8GB, latency_per_token < 5ms }
)
model SearchableCoder:
    embed: Tensor = randn([VOCAB, d_model]) * full([1], 0.02)
    blocks: [TransformerBlock; n_layers] = TransformerBlock(d_model, n_heads, n_kv_h
    norm: RMSNorm = RMSNorm(d_model)
    ...
```

CEP evaluates all feasible combinations through compilation:

```
Search space:  $4 \times 4 \times 3 \times 3 \times 3 = 432$  candidates
Compilation time:  $432 \times \sim 50\text{ms} = \sim 22$  seconds (single-threaded)

For each candidate:
1. Instantiate AST with specific dimensions
2. Run compilation oracle  $\rightarrow$  CompilationProfile
3. Check constraints (memory, latency)
4. Score:  $\text{param\_count} \times \text{roofline\_utilization} / \text{latency}$ 

Output: Ranked candidates with exact hardware metrics
```

No training. No surrogate models. No GPU. Just compilation analysis.

Mode 3: Joint Prune-Search (weights + architecture flexibility)

CEP can simultaneously prune and search — starting from a pre-trained model and finding the best pruned sub-architecture for a target:

Starting model: NSLCoder-50M (D=512, L=8, H=8, KV=4, FFN=1408)

Target: Fit in 4GB VRAM on RTX 4060 with <2ms/token latency

Joint search space:

- Which layers to keep ($2^8 = 256$ subsets)
- Per-layer: which heads to keep (combinatorial per layer)
- Per-layer: FFN width reduction ($1408 \rightarrow \{1408, 1024, 768, 512\}$)
- Weight importance guides the search (prune least important first)

Search strategy: Greedy pruning with compilation verification

1. Score all components by weight importance
2. Remove least important component
3. Recompile $\rightarrow$ check constraints
4. If constraints met: continue pruning
5. If constraints violated: backtrack and try next candidate
6. Terminate when target sparsity reached or Pareto-optimal

3. True Structural Pruning: Dead Code in the Binary

This is CEP's most distinctive capability. In PyTorch:

```
python
```

```
# "Pruning" in PyTorch
```

```
model.layer[5].attn.wq.weight[:, head_3_start:head_3_end] = 0 # zeroed but still th
```

```
# The model still runs 8-head attention – just with zeros in head 3
```

```
# Same FLOP count, same kernel launches, same memory
```

In NSL with CEP:

ns!

```
# After CEP determines head 3 in layer 5 should be pruned:
# The compiler generates a NEW model definition with 7 heads in layer 5

# Original:
model TransformerBlock:
    attn: GroupedQueryAttention(512, 8, 4, 0.1) # 8 Q heads, 4 KV heads

# CEP-pruned (layer 5 only):
model TransformerBlock_L5_Pruned:
    attn: GroupedQueryAttention(512, 7, 4, 0.1) # 7 Q heads, 4 KV heads
    # Wq is now [512, 448] instead of [512, 512]
    # FlashAttention grid is (n_q_tiles, batch×7, 1) instead of batch×8
    # SMEM layout is recomputed for 7 heads
```

What physically changes in the compiled binary:

Property	Unpruned (8 heads)	CEP-Pruned (7 heads)
Wq parameter size	512×512 = 262K params	512×448 = 229K params
Wq matmul FLOPs	537M	470M
FA kernel grid Y dimension	batch×8	batch×7
FA SMEM per CTA	64×64×2 = 8KB	64×64×2 = 8KB (same)
FA total kernel launches	batch×8 CTAs	batch×7 CTAs
Output projection Wo	[512, 512]	[448, 512]
Total attention FLOPs	2.14 GFLOP	1.87 GFLOP (12.5% less)

The pruned binary doesn't contain the zeroed head — it's gone from the executable. Smaller weight file, fewer kernel launches, less register pressure, tighter memory plan.

3.1 Per-Layer Heterogeneous Pruning

CEP can produce models where different layers have different widths — something extremely awkward in PyTorch (requires rewriting the model class) but natural in NSL:

```
CEP output for NSLCoder-50M targeting 4GB RAM:
```

```
Layer 0: 8/8 heads, FFN=1408 (critical for early representations)
Layer 1: 8/8 heads, FFN=1408
Layer 2: 7/8 heads, FFN=1024 (head 3 low importance, FFN oversized)
Layer 3: 6/8 heads, FFN=1024
Layer 4: 6/8 heads, FFN=768 (mid-layers are most prunable)
Layer 5: 6/8 heads, FFN=768
Layer 6: 7/8 heads, FFN=1024 (final layers need more capacity)
Layer 7: 8/8 heads, FFN=1408
```

```
Original: 48.8M params, 5.2 GB peak memory
```

```
Pruned: 31.2M params, 3.7 GB peak memory (36% smaller)
```

The compiler generates a different `TransformerBlock` variant for each layer — each with its own kernel configurations, tile sizes, and memory plan. The final binary contains 8 different block implementations, each optimized for its specific dimensions.

3.2 Dead Gradient Elimination for Pruned Training

When recovery fine-tuning a CEP-pruned model, the WRGA technique (from our previous research) activates automatically. The source AD generates backward passes that are shaped to the pruned architecture — no backward computation for removed heads. Combined with FASE, the fine-tuning of a CEP-pruned model has:

- No gradient paths for pruned components (physically absent)
- Fused accumulation for remaining gradients
- Memory plan optimized for the pruned topology

4. The Compilation-Guided Search Algorithm

4.1 Importance Scoring Without Gradients

CEP uses a novel **compile-time importance score** that doesn't require any forward or backward passes:

```
importance(head_h_in_layer_l) =
    weight_magnitude(Wq_h, Wk_h, Wv_h)    # how large are the projection weights
    × spectral_energy(Wq_h)                # what fraction of singular value mass
    × roofline_slack(layer_l)              # how memory-bound is this layer (free
compute)
    × position_factor(1 / L)               # early/late layers are more important
```


Each component is computable from weights + compilation analysis alone:

- **weight_magnitude:** Frobenius norm of the head's slice of W_q , W_k , W_v . Available from SafeTensors loading.
- **spectral_energy:** Ratio of top-k singular values to total, indicating how concentrated the head's representational capacity is. Heads with low spectral energy encode diffuse, low-information patterns.
- **roofoffline_slack:** From the cost model. Memory-bound heads "cost less" to keep (their compute is partially free) so they get an importance bonus.
- **position_factor:** Empirically, early and late layers are more sensitive to pruning. CEP uses $1.0 + 0.3 \times |2l/L - 1|$ (U-shaped curve).

4.2 Constraint-Driven Search

```
Algorithm: CEP-Search(model, weights, target_hw, constraints)

baseline = compile_oracle(model, target_hw, weights)
candidates = rank_all_components_by_importance(weights)

pruned_model = model
for component in candidates (ascending importance):
    trial = remove_component(pruned_model, component)
    profile = compile_oracle(trial, target_hw, weights_pruned)

    if satisfies_constraints(profile, constraints):
        pruned_model = trial
        if reached_target_sparsity():
            break
    else:
        # This component is too important or pushes past constraints
        continue

return pruned_model, compile_oracle(pruned_model, target_hw, weights_pruned)
```

Time complexity: $O(C \times T_{\text{compile}})$ where C = number of candidate components and $T_{\text{compile}} \approx 50\text{ms}$. For NSLCoder-50M with per-head and per-FFN-width granularity: $C \approx 100$ candidates $\rightarrow$ ~ 5 seconds total search time.

4.3 Architecture Search Space for LLMs

CEP defines a natural search space for transformer-based LLMs:

Parameter	Search Range	Granularity
d_model	[128, 2048]	Power of 2 or multiple of 64
n_layers	[2, 48]	Integer
n_heads	[2, 32]	Must divide d_model
n_kv_heads	[1, n_heads]	Must divide n_heads
head_dim	d_model/n_heads	Derived
d_ff	[d_model, 8×d_model]	Multiple of 64
activation	[relu, gelu, silu]	Categorical
norm_type	[layernorm, rmsnorm]	Categorical
vocab_size	fixed	Not searched

The compiler verifies each candidate's shape compatibility via shape algebra (M49) before evaluation — candidates with dimension mismatches are rejected without compilation.

5. Integration with NSL's Existing Infrastructure

CEP is uniquely powerful because it composes all of NSL's compiler subsystems:

NSL Subsystem	CEP Usage
Weight-aware compilation (M52)	Load pre-trained weights, compute importance scores
Roofline cost model (M37)	Per-op FLOP/byte analysis, latency estimation
Memory planner (M36)	Peak memory for each candidate architecture
Shape algebra (M49)	Verify dimension compatibility of candidates
WCET analysis (M53)	Worst-case latency bounds for safety-critical deployment
Fusion analysis (M31)	Adjusted performance after fusion for pruned architectures
Source AD (M40)	Generate pruned backward passes for recovery fine-tuning
Effect system (M51)	Verify pruned models maintain determinism/purity properties
GPU spec database	Target-specific evaluation (A100, H100, RTX 4090, etc.)

No other ML framework has all nine of these capabilities, let alone the ability to compose them in a single compilation pass.

6. NSL Language Surface

6.1 Pruning

```
ns!

# Load pre-trained model with weights
let model = NSLCoder.load("pretrained.nslm", weights="model.safetensors")

# CEP prunes to fit target hardware constraints
@cep_prune(
  target = rtx5070ti,
  sparsity = 0.3,           # remove 30% of parameters
  constraints = { peak_memory < 6GB, latency_per_token < 3ms },
  granularity = "head+ffn", # prune heads and FFN width
  preserve = ["blocks.0.*", "blocks.7.*"] # protect first and last layers
)
let pruned_model = model

# Recovery fine-tuning (WRGA + FASE activate automatically)
train(model=pruned_model, epochs=2):
  optimizer: AdamW(lr=1e-5)
  step(batch):
    let logits = pruned_model.forward_train(batch.input_ids, true)
    let loss = cross_entropy(logits, batch.labels)

model_save(pruned_model, "pruned_coder.nslm")
```

6.2 Architecture Search

ns!

```
# Define searchable model template
@cep_search(
    target = rtx5070ti,
    constraints = { params < 20M, peak_memory < 4GB, latency_per_token < 2ms },
    objective = "maximize param_efficiency" # params × roofline_util / latency
)
model SearchableLM:
    embed: Tensor = randn([VOCAB, @search(d_model, [256, 384, 512])])
    blocks: [TransformerBlock; @search(n_layers, [4, 6, 8])] = TransformerBlock(
        d_model,
        @search(n_heads, [4, 6, 8]),
        @search(n_kv_heads, [2, 3, 4]),
        @search(d_ff, [768, 1024, 1408]),
        0.1
    )
    norm: RMSNorm = RMSNorm(d_model)
```

6.3 CLI

```
bash
```

Pruning analysis

```
$ nsl build --weights model.safetensors --cep-prune --sparsity 0.3 --target h100 mod
```

```
=== CEP Pruning Report ===
```

```
Search time: 4.7 seconds (98 candidates evaluated)
```

```
Original: 48.8M params, 5.2GB peak, 6.9µs/token (H100)
```

```
Recommended pruning:
```

```
Layer 2: 7→6 heads, FFN 1408→1024 (-2.1M params)
```

```
Layer 3: 7→6 heads, FFN 1408→1024 (-2.1M params)
```

```
Layer 4: 8→6 heads, FFN 1408→768 (-3.8M params)
```

```
Layer 5: 8→6 heads, FFN 1408→768 (-3.8M params)
```

```
Layer 6: 8→7 heads, FFN 1408→1024 (-1.5M params)
```

```
Pruned: 35.5M params (27% reduction), 3.8GB peak, 4.8µs/token
```

```
All constraints satisfied.
```

```
Binary size: 142MB → 103MB
```

```
Kernel launches per forward: 127 → 98
```

Architecture search

```
$ nsl check --cep-search --target rtx4060 searchable_model.nsl
```

```
=== CEP Architecture Search Report ===
```

```
Search space: 324 candidates
```

```
Feasible (constraints met): 47 candidates
```

```
Compilation time: 16.2 seconds
```

```
Top 3 architectures:
```

```
1. d=384, L=6, H=6, KV=3, FFN=1024 → 10.1M params, 2.1GB, 1.8ms/tok, util=0.72
```

```
2. d=384, L=8, H=6, KV=3, FFN=768 → 11.3M params, 2.4GB, 1.9ms/tok, util=0.68
```

```
3. d=256, L=8, H=4, KV=2, FFN=1024 → 7.2M params, 1.6GB, 1.2ms/tok, util=0.71
```

7. Why PyTorch Cannot Do This

7.1 No Compilation Oracle

PyTorch has no equivalent of `nsl check --perf`. To evaluate a candidate architecture's hardware performance, you must:

1. Instantiate the model in Python
2. Create dummy inputs
3. Profile with `torch.profiler` or `nsys` (requires GPU, takes seconds)
4. Parse the profile output

This is 100-1000× slower than CEP's compilation analysis and requires a GPU.

7.2 No True Structural Pruning

When PyTorch prunes a head, the weight tensor is zeroed or masked. The matmul kernel still runs at the original dimensions. Getting true structural pruning requires manually rewriting the model code — adjusting Linear layer dimensions, attention head counts, and all downstream shape dependencies. This is error-prone and not automated.

NSL's compiler recompiles the model with new dimensions automatically. The shape algebra verifies all downstream dimensions are consistent. The PTX kernels are regenerated for the new sizes.

7.3 No Cross-Concern Evaluation

PyTorch cannot jointly evaluate FLOP efficiency + memory plan + fusion opportunities + latency + WCET for a candidate architecture, because these analyses live in different tools (`torch.profiler`, `memory_stats`, `compile logs`, `nsight`). CEP evaluates all of them in a single compilation pass because they are all part of the same compiler infrastructure.

7.4 No Weight-Informed Hardware Analysis

PyTorch's profiling doesn't consider weight values. CEP uses weight spectral analysis to inform hardware decisions — e.g., a head with low-rank weights may benefit from LoRA-style replacement rather than pruning, and the cost model can evaluate both options.

8. Performance Projections

8.1 CEP Pruning Results (Projected, NSLCoder-50M)

Sparsity	Params	Peak Mem	Latency (H100)	Binary Size	Search Time
0% (baseline)	48.8M	5.2 GB	6.9 μ s/tok	195 MB	—
20%	39.0M	4.2 GB	5.5 μ s/tok	156 MB	3.2s
30%	34.2M	3.7 GB	4.8 μ s/tok	137 MB	4.7s
50%	24.4M	2.6 GB	3.5 μ s/tok	98 MB	7.1s

8.2 CEP Search Results (Projected, 10M-param target on RTX 4060)

Method	Search Cost	GPU Required	Candidates Evaluated	Hardware Accuracy
Supernet NAS	2-5 GPU-days	Yes	~1000 (trained)	Approximate
Zero-cost proxy NAS	1-2 GPU-hours	Yes	~5000	Approximate
LLM-NAS	Minutes	No (LLM API)	~200	Approximate
CEP	16 seconds	No	324 (compiled)	Exact

9. Implementation Plan

Component	Location	LOC (est.)	Description
CEP oracle	<code>ns1-codegen/src/cep_oracle.rs</code>	500	Compose cost model + memory planner + fusion analysis
Weight importance	<code>ns1-codegen/src/cep_importance.rs</code>	400	Spectral + magnitude + roofline importance scoring
Pruning search	<code>ns1-codegen/src/cep_prune.rs</code>	600	Greedy + constraint-driven pruning algorithm
Architecture search	<code>ns1-codegen/src/cep_search.rs</code>	500	Enumerate + compile + rank candidates
Model rewriter	<code>ns1-codegen/src/cep_rewrite.rs</code>	700	AST rewriting for pruned architectures
CLI integration	<code>ns1-cli/src/main.rs</code>	150	<code>--cep-prune</code> and <code>--cep-search</code> flags

Dependencies: cost_model.rs, memory_planner.rs, weight_aware.rs, shape_algebra.rs, wcet.rs, fusion.rs

10. Conclusion

CEP completes our suite of compiler-native training and optimization techniques:

Technique	Domain	Core Insight
WRGA	PEFT	Adapter placement is a compilation decision
CSHA	Attention	Fusion boundaries are compiler-removable
FASE	Gradient Accumulation	Accumulation + optimizer can be fused at compile time
PCA	Sequence Packing	Document masks can be baked into kernel control flow
CEP	Pruning + NAS	The compiler IS the evaluation oracle

CEP's central insight: **architecture evaluation is a compilation problem**. The NSL compiler already contains everything needed to evaluate a neural network architecture's hardware performance — FLOP counting, roofline analysis, memory planning, latency estimation, fusion analysis, weight-aware optimization. CEP simply surfaces this capability as a user-facing tool, replacing the expensive runtime evaluation (training, profiling, surrogate modeling) that existing frameworks require with millisecond-cost compilation analysis.

The result is a pruning and architecture search system that is 100-1000× faster than existing approaches, requires no GPU for the search phase, produces exactly optimized binaries (not zeroed-weight approximations), and automatically composes with all other NSL compiler optimizations (CSHA attention fusion, WRGA fine-tuning, FASE gradient accumulation) for the pruned architecture.

References

1. Ma, X., Fang, G., Wang, X. "LLM-Pruner: On the Structural Pruning of Large Language Models." NeurIPS 2023.
2. Muralidharan, S., et al. "Compact Language Models via Pruning and Knowledge Distillation." NeurIPS 2024.
3. Sun, M., et al. "A Simple and Effective Pruning Approach for Large Language Models (Wanda)." ICML 2024.
4. Abdelfattah, M., et al. "Zero-Cost Proxies for Lightweight NAS." ICLR 2021.
5. Zhu, H., et al. "LLM-NAS: LLM-driven Hardware-Aware Neural Architecture Search." arXiv 2510.01472, 2025.
6. Frankle, J., Carbin, M. "The Lottery Ticket Hypothesis." ICLR 2019.
7. Williams, S., Waterman, A., Patterson, D. "Roofline: An Insightful Visual Performance Model." CACM 2009.